

Time-Series Convolution

In my applications, f is a waveform time series that is convolved with a pulse g . The equation for the their convolution is

$$y(t) = f(t) \star g(t) \equiv \int_{-\infty}^{\infty} dt' f(t') g(t - t'). \quad (1)$$

Both f and g are functions of time, and their zero times are coupled through the term $g(t - t')$. From Equation (1), it is easy to show that I can (and do) choose the time zero for y and f to be the same. In the applications discussed below, the zero time for $g(t)$ can be the same as or less than the zero time for $f(t)$.

To calculate the convolution, we discretize both f and g and replace the integral with a sum. In this discussion I will use $\delta t = 0.02s$ as the digitizing interval for y , f , and g . I multiply the sum by δt , which is not what is done in a “discrete convolution” — which also does not take into account a difference in zero time between $f(t)$ and $g(t)$.

I discuss two applications of convolution: both have the same $f(t)$ but different $g(t)$.

- $f(t)$ is a synthetic waveform produced using Haskell matrices or WKBJ where vertical lines of calculated polarities and amplitudes are drawn at phase arrival times. For these examples, I use Haskell matrices to produce a vertical-component waveform for an incident P -wave at an angle of 20° with the vertical through a 2-layer crust. There are $n_w = 2048$ points in the discretized f . (The large number is chosen to minimize wraparound.) The duration is $t_w = \delta t [n_w - 1]$. I choose the first point in f to be at $t = 0$, so $f(t) = f(t)H(t)H(t_w - t)$, where

$$H(t) = \begin{cases} 1, & \text{if } t > 0; \\ 0, & \text{if } t < 0. \end{cases}$$

- $g(t)$ is a pulse waveform with a far smaller duration than $f(t)$. The pulse can be either (1) an approximation of a P arrival so that the output of the convolution potentially models the data¹, or (2) a time-symmetric shape with a maximum at $t = 0$ to smooth out the Gibbs phenomenon that often accompanies arrivals in synthetics. I have used (2) for synthetics that generate receiver functions.

For this discussion, I use a “Brune” pulse for (1): $g(t) = U_0 H(t) t e^{-\frac{t}{\kappa}} H(t_p - t)$, where $\kappa = 0.1s$, $t_p = 1.26s$, and U_0 is a constant that only affects the amplitude so is of no interest here.

For (2), I use a triangle function produced by `fg` in `SAC`:

`fg triangle npts 8 delta 0.02 begin -0.08.`

Note that the Brune pulse starts at the same time as f , while the triangle pulse starts at a negative time with a maximum at the start time for f . If the total time for the pulse is t_p , the general form for the pulse is $g(t) = g(t)H(t - t_1)H(t_2 - t)$, where for the Brune

¹e.g. James, D. E. and Snoke, J. A. (1994). Structure and tectonics in the region of flat subduction beneath central Peru. Part I: Crust and uppermost mantle; *J. Geophys. Res.*, **99**, 6899D-6912.

Time-Series Convolution

pulse $t_1 = 0$, $t_2 = t_p$ and for the triangle pulse $t_1 = -0.08s$ $t_2 = 0.06s = t_p - t_1$. For either pulse, $t_p = \delta t [n_p - 1]$.

Given the above forms for f and g , Equation (1) can be written

$$y(t) = \int_0^{t_w} dt' f(t') g(t-t') H(t-t'-t_1) H(t_2-t+t'). \quad (2)$$

For the Brune pulse, $t_1 = 0$, so t cannot be negative, but for the triangle pulse the lower bound for t is $-0.06s$. In Fortran, arrays are stored in the computer using positive integers, so to simplify the bookkeeping it is best to avoid negative times when we discretize Equation (2). We can avoid negative times if we look at $y(\tau) = y(t - t_1)$. Equation (2) then reads

$$y(\tau) = y(t - t_1) = \int_0^{t_w} dt' f(t') g(\tau + t_1 - t') H(\tau - t') H(t_2 - \tau - t_1 + t'). \quad (3)$$

The discretized equation for Equation (3) is

$$y_i = \delta t \sum_{j=1}^{n_w} f_j g_{i-j-j_1}, \quad (4)$$

where $j_1 = -[t_1/\delta t] + 1$ and i runs from 1 to $n_w + n_p - 1$.

The following Fortran code produces the correct result for the convolution for either pulse:

```
j_1 = -nint(b_p/delta)+1
do i=1,n_w+n_p-1
  temp = 0.0
  do j=1,n_w
    if (i.ge.(j-j_1) .and. n_p.ge.(i-j+j_1))
1      temp = temp + waveform(j)*pulse(i-j+j_1)
  end do
  conv(i) = delta*temp
end do
```

where $b_p = t_1$, $\delta t = \delta t$, and $\text{conv}(i) = y_i$.

Plots for the two pulse waveforms are shown in Figure 1, and the results of the convolution near the first synthetic arrival are shown in Figure 2 for both the Brune pulse the triangle pulse. The Gibbs phenomenon is quite pronounced at the arrival in the unfiltered synthetic. Note that the peaks for the arrival in Figure 2 are at the same time for the triangle-pulse convolution and the synthetic and the Brune-pulse convolution starts at the peak of the unfiltered-synthetic arrival time. For display purposes, I time-shifted the waveform so that the first arrival is at five seconds into the record. Hence the zero time for the display and the convolution calculation are not the same.

When I use the SAC convolution for my two runs, I get the "correct" result for the Brune pulse but a time shift of 0.08 seconds for the triangle pulse (which starts at $-0.08s$). The

Time-Series Convolution

convolution of the triangle pulse with itself is also time-shifted 0.08s. If I use the (new) SAC CONVOLVE option “amplitude on”, I get the same amplitudes for SAC convolution output as with my program.

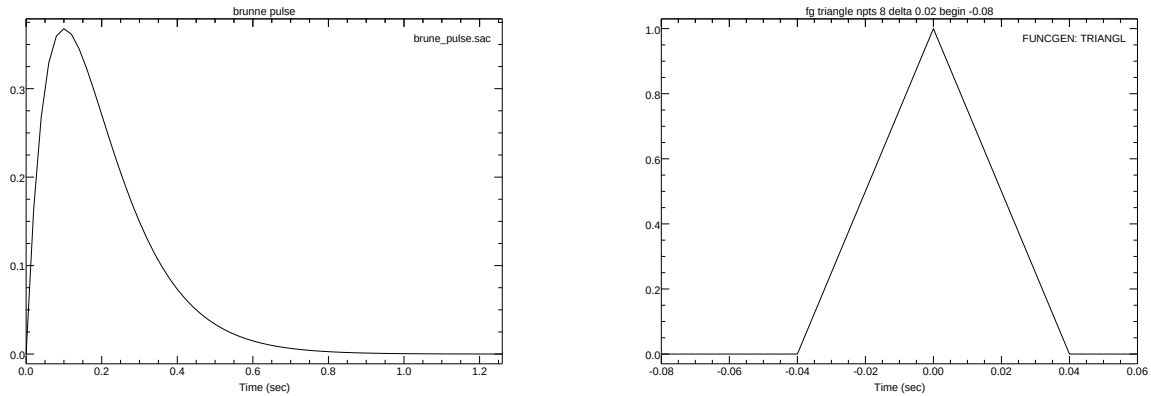


Figure 1: The two source pulses used for my convolution applications.

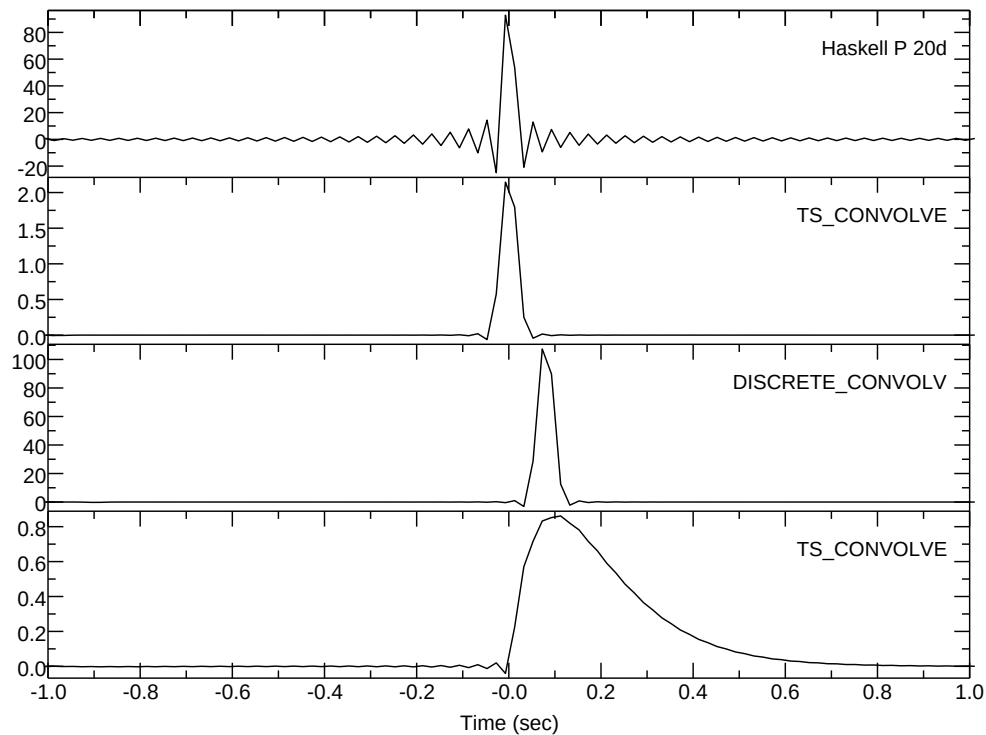


Figure 2: From top to bottom: unfiltered synthetic, time-series convolution with triangle pulse, discrete convolution with triangle pulse, time-series convolution with Brune pulse.